

Alocação de Registradores



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismaily
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)

Introdução

- A IR e a seleção de instruções assumiram que o número de registradores era infinito.

Introdução

- A IR e a seleção de instruções assumiram que o número de registradores era infinito.
- Objetivo

Introdução

- A IR e a seleção de instruções assumiram que o número de registradores era infinito.
- Objetivo
 - Atribuir registradores físicos (da máquina) para os temporários usados nas instruções.

Introdução

- A IR e a seleção de instruções assumiram que o número de registradores era infinito.
- Objetivo
 - Atribuir registradores físicos (da máquina) para os temporários usados nas instruções.
 - Se possível, atribuir a fonte e o destino de MOVES para o mesmo registrador.

Introdução

- A IR e a seleção de instruções assumiram que o número de registradores era infinito.
- Objetivo
 - Atribuir registradores físicos (da máquina) para os temporários usados nas instruções.
 - Se possível, atribuir a fonte e o destino de MOVES para o mesmo registrador.
 - Eliminando os MOVES inúteis.

Introdução

- Grafo de interferência (IG):

Introdução

- Grafo de interferência (IG):
 - Temos arestas entre t_1 e t_2 se eles não podem ocupar o mesmo registrador.

Introdução

- Grafo de interferência (IG):
 - Temos arestas entre t_1 e t_2 se eles não podem ocupar o mesmo registrador.
 - Live ranges têm intersecção.

Introdução

- Grafo de interferência (IG):
 - Temos arestas entre t_1 e t_2 se eles não podem ocupar o mesmo registrador.
 - Live ranges têm intersecção.
 - Restrições da arquitetura

Introdução

- Grafo de interferência (IG):
 - Temos arestas entre t_1 e t_2 se eles não podem ocupar o mesmo registrador.
 - Live ranges têm intersecção.
 - Restrições da arquitetura
 - $a = a + b$ não pode ser atribuído ao r_{12} .

Introdução

- Grafo de interferência (IG):
 - Temos arestas entre t_1 e t_2 se eles não podem ocupar o mesmo registrador.
 - Live ranges têm intersecção.
 - Restrições da arquitetura
 - $a = a + b$ não pode ser atribuído ao r_{12} .
- O problema se transforma em um problema de coloração de grafos.

Coloração do IG

- Queremos colorir o IG utilizando o mínimo de cores possíveis, de maneira que nenhum par de vértices conectados por uma aresta tenham a mesma cor

Coloração do IG

- Queremos colorir o IG utilizando o mínimo de cores possíveis, de maneira que nenhum par de vértices conectados por uma aresta tenham a mesma cor
 - Coloração de vértices.

Coloração do IG

- Queremos colorir o IG utilizando o mínimo de cores possíveis, de maneira que nenhum par de vértices conectados por uma aresta tenham a mesma cor
 - Coloração de vértices.
 - As cores representam os registradores.

Coloração do IG

- Queremos colorir o IG utilizando o mínimo de cores possíveis, de maneira que nenhum par de vértices conectados por uma aresta tenham a mesma cor
 - Coloração de vértices.
 - As cores representam os registradores.
 - Considerando que a máquina tem k registradores.

Coloração do IG

- Queremos colorir o IG utilizando o mínimo de cores possíveis, de maneira que nenhum par de vértices conectados por uma aresta tenham a mesma cor
 - Coloração de vértices.
 - As cores representam os registradores.
 - Considerando que a máquina tem k registradores.
 - Se encontrarmos uma k -coloração para o IG, essa coloração é uma alocação válida dos registradores.

Coloração do IG

- E se não existe uma k -coloração?

Coloração do IG

- E se não existe uma k -coloração?
 - Então teremos que colocar alguns dos temporários ou variáveis na memória.

Coloração do IG

- E se não existe uma k -coloração?
 - Então teremos que colocar alguns dos temporários ou variáveis na memória.
 - Operação conhecida como *spilling*.

Coloração do IG

- E se não existe uma k -coloração?
 - Então teremos que colocar alguns dos temporários ou variáveis na memória.
 - Operação conhecida como *spilling*.
- Coloração de vértices é um problema NP-Completo.

Coloração do IG

- E se não existe uma k -coloração?
 - Então teremos que colocar alguns dos temporários ou variáveis na memória.
 - Operação conhecida como *spilling*.
- Coloração de vértices é um problema NP-Completo.
 - Logo, alocação de registradores também é.

Coloração do IG

- E se não existe uma k -coloração?
 - Então teremos que colocar alguns dos temporários ou variáveis na memória.
 - Operação conhecida como *spilling*.
- Coloração de vértices é um problema NP-Completo.
 - Logo, alocação de registradores também é.
- Existe uma aproximação linear que traz bons resultados.

Coloração por Simplificação

- Dividir em 5 fases:

Coloração por Simplificação

- Dividir em 5 fases:
 - 1 Build:

Coloração por Simplificação

- Dividir em 5 fases:
 - ① Build:
 - Construir o IG

Coloração por Simplificação

- Dividir em 5 fases:
 - ① Build:
 - Construir o IG
 - Usa a análise de longevidade.

Coloração por Simplificação

- Dividir em 5 fases:
 - ① Build:
 - Construir o IG
 - Usa a análise de longevidade.
 - ② Simplify

Coloração por Simplificação

- Dividir em 5 fases:
 - 1 Build:
 - Construir o IG
 - Usa a análise de longevidade.
 - 2 Simplify
 - Heurística: Suponha que o grafo G tenha um vértice v com menos de k vizinhos.

Coloração por Simplificação

- Dividir em 5 fases:

- ① Build:

- Construir o IG
 - Usa a análise de longevidade.

- ② Simplify

- Heurística: Suponha que o grafo G tenha um vértice v com menos de k vizinhos.
 - k é o número de registradores.

Coloração por Simplificação

- Dividir em 5 fases:

- ① Build:

- Construir o IG
 - Usa a análise de longevidade.

- ② Simplify

- Heurística: Suponha que o grafo G tenha um vértice v com menos de k vizinhos.
 - k é o número de registradores.
 - Faça $G' = G - \{v\}$

Coloração por Simplificação

- Dividir em 5 fases:

- ① Build:

- Construir o IG
 - Usa a análise de longevidade.

- ② Simplify

- Heurística: Suponha que o grafo G tenha um vértice v com menos de k vizinhos.
 - k é o número de registradores.
 - Faça $G' = G - \{v\}$
 - Se G' pode ser colorido com k cores, G também pode.

Coloração por Simplificação

- ② Simplify - Leva a um algoritmo recursivo (pilha).

Coloração por Simplificação

- ② Simplify - Leva a um algoritmo recursivo (pilha).
 - Repetidamente:

Coloração por Simplificação

- ② Simplify - Leva a um algoritmo recursivo (pilha).
 - Repetidamente:
 - Remova vértices com grau menor que k

Coloração por Simplificação

- ② Simplify - Leva a um algoritmo recursivo (pilha).
 - Repetidamente:
 - Remova vértices com grau menor que k
 - Coloque na pilha

Coloração por Simplificação

- ② Simplify - Leva a um algoritmo recursivo (pilha).
 - Repetidamente:
 - Remova vértices com grau menor que k
 - Coloque na pilha
 - Cada remoção diminui o grau dos vértices em G , dando oportunidades para novas remoções.

Coloração por Simplificação

③ Spill:

Coloração por Simplificação

③ Spill:

- Em algum momento não temos um vértice com grau $< k$

Coloração por Simplificação

③ Spill:

- Em algum momento não temos um vértice com grau $< k$
- A heurística falha.

Coloração por Simplificação

③ Spill:

- Em algum momento não temos um vértice com grau $< k$
- A heurística falha.
- Temos que marcar algum vértice para spill.

Coloração por Simplificação

③ Spill:

- Em algum momento não temos um vértice com grau $< k$
- A heurística falha.
- Temos que marcar algum vértice para spill.
- A escolha desse vértice é também uma heurística.

Coloração por Simplificação

③ Spill:

- Em algum momento não temos um vértice com grau $< k$
- A heurística falha.
- Temos que marcar algum vértice para spill.
- A escolha desse vértice é também uma heurística.
 - vértice que reduza o grau do maior número de outros vértices.

Coloração por Simplificação

③ Spill:

- Em algum momento não temos um vértice com grau $< k$
- A heurística falha.
- Temos que marcar algum vértice para spill.
- A escolha desse vértice é também uma heurística.
 - vértice que reduza o grau do maior número de outros vértices.
 - vértice com menos custo relacionado as operações de memória.

Coloração por Simplificação

④ Select:

Coloração por Simplificação

④ Select:

- Atribui as cores

Coloração por Simplificação

④ Select:

- Atribui as cores
- Reconstrói o grafo G adicionando os vértices na ordem determinada pela pilha.

Coloração por Simplificação

④ Select:

- Atribui as cores
- Reconstrói o grafo G adicionando os vértices na ordem determinada pela pilha.
- Quando adicionamos um vértice, devemos ter uma cor para ele dado o critério de seleção usado para remover.

Coloração por Simplificação

④ Select:

- Atribui as cores
- Reconstrói o grafo G adicionando os vértices na ordem determinada pela pilha.
- Quando adicionamos um vértice, devemos ter uma cor para ele dado o critério de seleção usado para remover.
- Isso não vale para os vértices empilhados marcados como spill.

Coloração por Simplificação

④ Select:

- Atribui as cores
- Reconstrói o grafo G adicionando os vértices na ordem determinada pela pilha.
- Quando adicionamos um vértice, devemos ter uma cor para ele dado o critério de seleção usado para remover.
- Isso não vale para os vértices empilhados marcados como spill.
 - Se todos os vizinhos já usarem k cores, não adicionamos no grafo.

Coloração por Simplificação

④ Select:

- Atribui as cores
- Reconstrói o grafo G adicionando os vértices na ordem determinada pela pilha.
- Quando adicionamos um vértice, devemos ter uma cor para ele dado o critério de seleção usado para remover.
- Isso não vale para os vértices empilhados marcados como spill.
 - Se todos os vizinhos já usarem k cores, não adicionamos no grafo.
 - Continua o processo.

Coloração por Simplificação

⑤ Start Over:

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:
 - $c_1 = M[c_{alloc}]$

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:
 - $c_1 = M[c_{alloc}]$
 - $a = b + c_1$ e para o outro caso $c = d + 1$, teremos:

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:
 - $c_1 = M[c_{alloc}]$
 - $a = b + c_1$ e para o outro caso $c = d + 1$, teremos:
 - $c_2 = d + 1$

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:
 - $c_1 = M[c_{alloc}]$
 - $a = b + c_1$ e para o outro caso $c = d + 1$, teremos:
 - $c_2 = d + 1$
 - $M[c_{alloc}] = c_2$

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:
 - $c_1 = M[c_{alloc}]$
 - $a = b + c_1$ e para o outro caso $c = d + 1$, teremos:
 - $c_2 = d + 1$
 - $M[c_{alloc}] = c_2$
- Isso gera novos temporários, ou seja, para cada uso (ou definição), um novo temporário é criado

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:
 - $c_1 = M[c_{alloc}]$
 - $a = b + c_1$ e para o outro caso $c = d + 1$, teremos:
 - $c_2 = d + 1$
 - $M[c_{alloc}] = c_2$
- Isso gera novos temporários, ou seja, para cada uso (ou definição), um novo temporário é criado
 - Com live ranges mais curtas.

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:
 - $c_1 = M[c_{alloc}]$
 - $a = b + c_1$ e para o outro caso $c = d + 1$, teremos:
 - $c_2 = d + 1$
 - $M[c_{alloc}] = c_2$
- Isso gera novos temporários, ou seja, para cada uso (ou definição), um novo temporário é criado
 - Com live ranges mais curtas.
- O algoritmo é repetido desde a construção do IG.

Coloração por Simplificação

⑤ Start Over:

- Pode ser que o Select não consiga atribuir uma cor a algum vértice.
- Reescrever o programa para pegar esse valor da memória antes de cada uso e armazená-lo de volta após o uso. Se tem algo como $a = b + c$ e $c = d + 1$, e c vai para memória (spilling), então o código ficará:
 - $c_1 = M[c_{alloc}]$
 - $a = b + c_1$ e para o outro caso $c = d + 1$, teremos:
 - $c_2 = d + 1$
 - $M[c_{alloc}] = c_2$
- Isso gera novos temporários, ou seja, para cada uso (ou definição), um novo temporário é criado
 - Com live ranges mais curtas.
- O algoritmo é repetido desde a construção do IG.
- O processo acaba quando Select tiver sucesso para todos os vértices.

Colocação por Simplificação

Principais fases:

Colocação por Simplificação

Principais fases:

- 1 Build

Colocação por Simplificação

Principais fases:

- ① Build
- ② Simplify

Colocação por Simplificação

Principais fases:

- ① Build
- ② Simplify
- ③ Spill

Colocação por Simplificação

Principais fases:

- ① Build
- ② Simplify
- ③ Spill
- ④ Select

Colocação por Simplificação

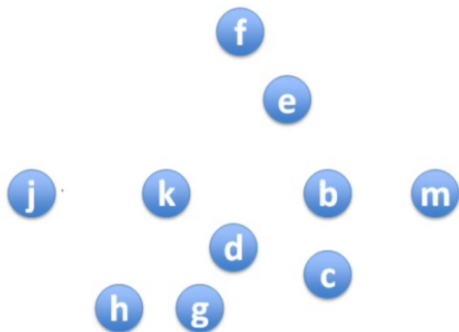
Principais fases:

- ① Build
- ② Simplify
- ③ Spill
- ④ Select
- ⑤ Start Over

Exemplo

- Suponha que temos 4 registradores

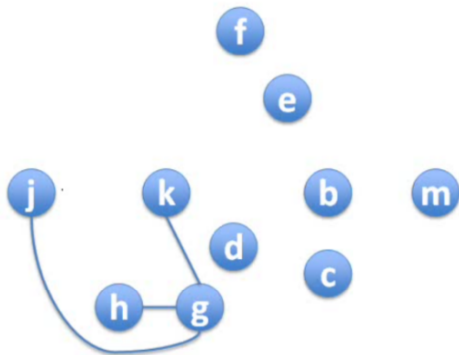
```
live-in: k j
  g := mem[j+12]
→ h := k - 1
  f := g * h
  e := mem[j+8]
  m := mem[j+16]
  b := mem[f]
  c := e + 8
  d := c
  k := m + 4
  j := b
live-out: d k j
```



Exemplo

- Suponha que temos 4 registradores

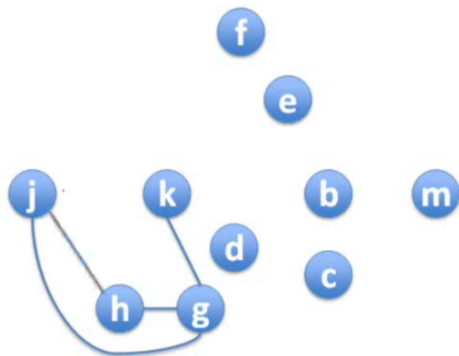
```
live-in: k j
  g := mem[j+12]
  h := k - 1
→ f := g * h
  e := mem[j+8]
  m := mem[j+16]
  b := mem[f]
  c := e + 8
  d := c
  k := m + 4
  j := b
live-out: d k j
```



Exemplo

- Suponha que temos 4 registradores

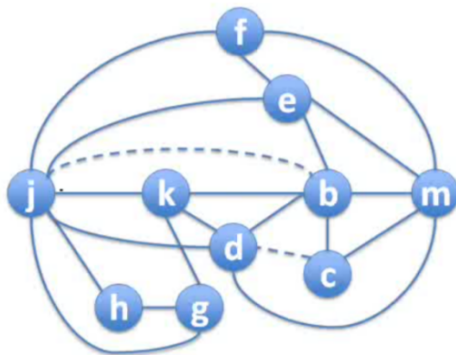
```
live-in: k j
  g := mem[j+12]
  h := k - 1
  f := g * h
→ e := mem[j+8]
  m := mem[j+16]
  b := mem[f]
  c := e + 8
  d := c
  k := m + 4
  j := b
live-out: d k j
```



Exemplo

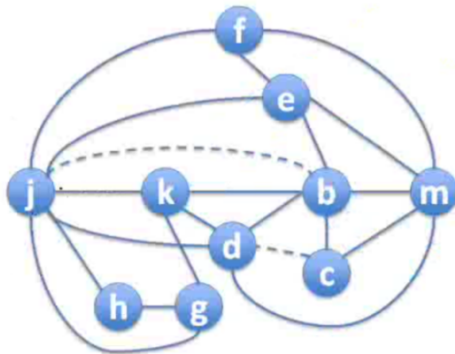
- Suponha que temos 4 registradores

```
live-in: k j  
  g := mem[j+12]  
  h := k - 1  
  f := g * h  
  e := mem[j+8]  
  m := mem[j+16]  
  b := mem[f]  
  c := e + 8  
  d := c  
  k := m + 4  
  j := b  
live-out: d k j
```



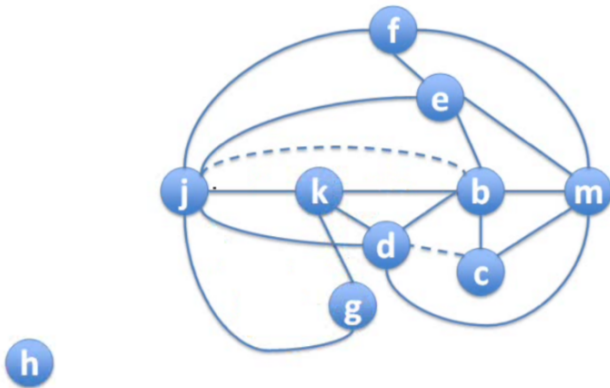
Exemplo

- Removendo h, g , k



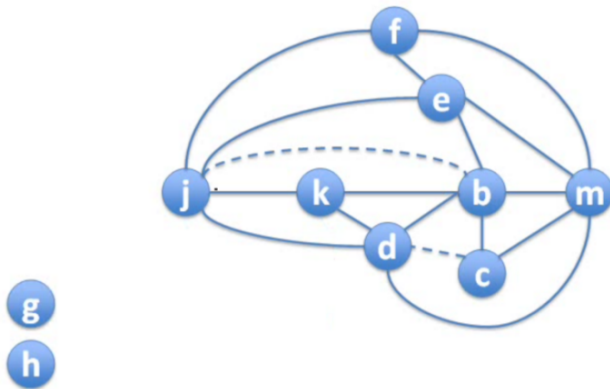
Exemplo

- Removendo h, g , k



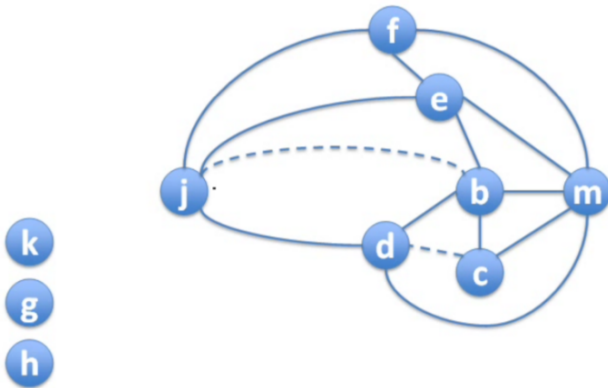
Exemplo

- Removendo h, g , k



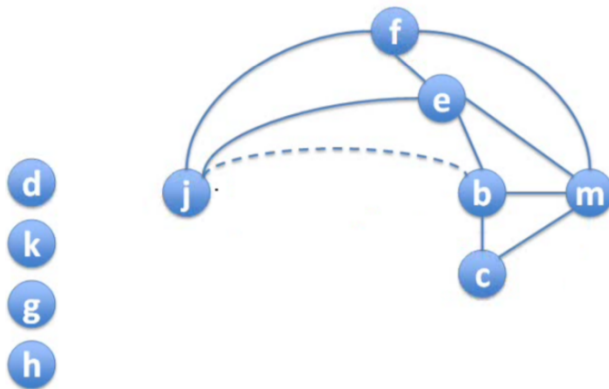
Exemplo

- Removendo h, g , k



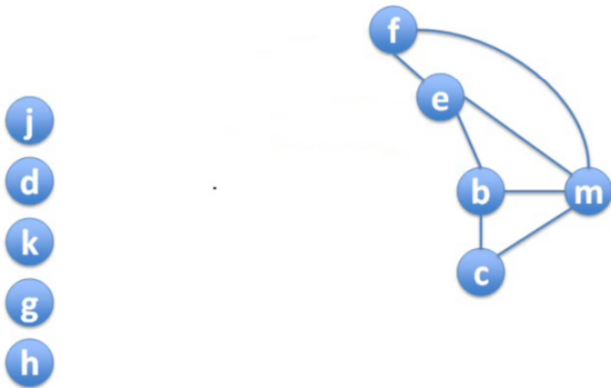
Exemplo

- Removendo mais nós



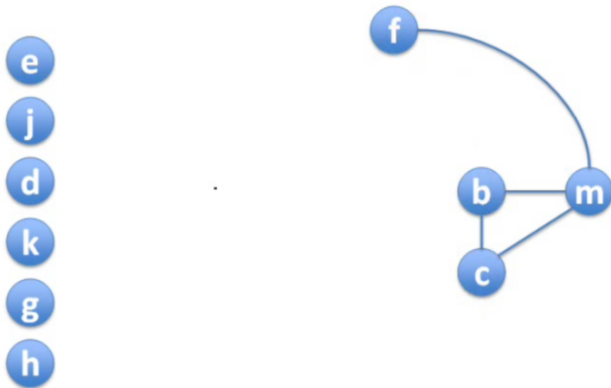
Exemplo

- Removendo mais nós

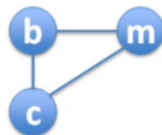


Exemplo

- Removendo mais nós



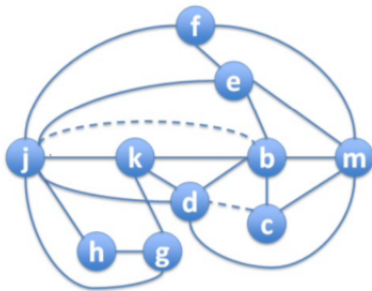
Exemplo



Exemplo

m
c
b
f
e
j
d
k
g
h

Exemplo – Colorindo



Coalescing

- Eliminar cópias (MOVES) redundantes usando o IG.

Coalescing

- Eliminar cópias (MOVES) redundantes usando o IG.
 - Se não existirem arestas entre os vértices das variáveis de uma instrução de cópia ela pode ser eliminada.

Coalescing

- Eliminar cópias (MOVES) redundantes usando o IG.
 - Se não existirem arestas entre os vértices das variáveis de uma instrução de cópia ela pode ser eliminada.
- Os vértices fonte e destino da cópia são unidos (*coalesced*) em um só vértice.

Coalescing

- Eliminar cópias (MOVES) redundantes usando o IG.
 - Se não existirem arestas entre os vértices das variáveis de uma instrução de cópia ela pode ser eliminada.
- Os vértices fonte e destino da cópia são unidos (*coalesced*) em um só vértice.
- O conjunto de arestas do novo vértice é a união das arestas dos dois anteriores.

Coalescing

- O efeito é sempre benéfico?

Coalescing

- O efeito é sempre benéfico?
 - Qualquer instrução de cópia sem arestas no IG poderia ser eliminada

Coalescing

- O efeito é sempre benéfico?
 - Qualquer instrução de cópia sem arestas no IG poderia ser eliminada
 - Pode tornar o processo de alocação mais complicado.

Coalescing

- O efeito é sempre benéfico?
 - Qualquer instrução de cópia sem arestas no IG poderia ser eliminada
 - Pode tornar o processo de alocação mais complicado.
 - Porque?

Coalescing

- O efeito é sempre benéfico?
 - Qualquer instrução de cópia sem arestas no IG poderia ser eliminada
 - Pode tornar o processo de alocação mais complicado.
 - Porque?
- O vértice resultante é mais restritivo que os anteriores.

Coalescing

- O efeito é sempre benéfico?
 - Qualquer instrução de cópia sem arestas no IG poderia ser eliminada
 - Pode tornar o processo de alocação mais complicado.
 - Porque?
- O vértice resultante é mais restritivo que os anteriores.
 - Seu grau aumenta.

Coalescing

- O efeito é sempre benéfico?
 - Qualquer instrução de cópia sem arestas no IG poderia ser eliminada
 - Pode tornar o processo de alocação mais complicado.
 - Porque?
- O vértice resultante é mais restritivo que os anteriores.
 - Seu grau aumenta.
 - Pode se tornar $\geq k$

Coalescing

- O efeito é sempre benéfico?
 - Qualquer instrução de cópia sem arestas no IG poderia ser eliminada
 - Pode tornar o processo de alocação mais complicado.
 - Porque?
- O vértice resultante é mais restritivo que os anteriores.
 - Seu grau aumenta.
 - Pode se tornar $\geq k$
- Um grafo k -colorável antes do *coalescing* pode se tornar não k -colorável após uma operação de *coalescing*.

Coalescing

- Devemos tomar cuidados.

Coalescing

- Devemos tomar cuidados.
 - Executar *coalescing* somente quando for seguro.

Coalescing

- Devemos tomar cuidados.
 - Executar *coalescing* somente quando for seguro.
 - Temos duas estratégias.

Coalescing

- Devemos tomar cuidados.
 - Executar *coalescing* somente quando for seguro.
 - Temos duas estratégias.
- Briggs:

Coalescing

- Devemos tomar cuidados.
 - Executar *coalescing* somente quando for seguro.
 - Temos duas estratégias.
- Briggs:
 - a e b podem ser unidos se o vértice resultante ab tiver menos do que k vizinhos com grau significativo ($\geq k$)

Coalescing

- Devemos tomar cuidados.
 - Executar *coalescing* somente quando for seguro.
 - Temos duas estratégias.
- Briggs:
 - a e b podem ser unidos se o vértice resultante ab tiver menos do que k vizinhos com grau significativo ($\geq k$)
 - Garante que o grafo continua k -colorável. Porque?

Coalescing

- Devemos tomar cuidados.
 - Executar *coalescing* somente quando for seguro.
 - Temos duas estratégias.
- Briggs:
 - a e b podem ser unidos se o vértice resultante ab tiver menos do que k vizinhos com grau significativo ($\geq k$)
 - Garante que o grafo continua k -colorável. Porque?
 - Após a simplificação remover todos os vértices não-significativos, sobram menos do que k vizinhos para o vértice ab

Coalescing

- Devemos tomar cuidados.
 - Executar *coalescing* somente quando for seguro.
 - Temos duas estratégias.
- Briggs:
 - a e b podem ser unidos se o vértice resultante ab tiver menos do que k vizinhos com grau significativo ($\geq k$)
 - Garante que o grafo continua k -colorável. Porque?
 - Após a simplificação remover todos os vértices não-significativos, sobram menos do que k vizinhos para o vértice ab
 - Logo, ele pode ser removido.

Coalescing

- George:

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :
 - t interfere com b .

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :
 - t interfere com b .
 - ou t tem grau insignificante ($< k$)

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :
 - t interfere com b .
 - ou t tem grau insignificante ($< k$)
 - Por que é segura?

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :
 - t interfere com b .
 - ou t tem grau insignificante ($< k$)
 - Por que é segura?
 - Seja S o conjunto de vizinhos insignificantes de a em G

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :
 - t interfere com b .
 - ou t tem grau insignificante ($< k$)
 - Por que é segura?
 - Seja S o conjunto de vizinhos insignificantes de a em G
 - Sem o *coalescing*, todos poderiam ser removidos, gerando um grafo G_1 .

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :
 - t interfere com b .
 - ou t tem grau insignificante ($< k$)
 - Por que é segura?
 - Seja S o conjunto de vizinhos insignificantes de a em G
 - Sem o *coalescing*, todos poderiam ser removidos, gerando um grafo G_1 .
 - Fazendo o *coalescing*, todos os vértices de S também poderão ser removidos, criando G_2 .

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :
 - t interfere com b .
 - ou t tem grau insignificante ($< k$)
 - Por que é segura?
 - Seja S o conjunto de vizinhos insignificantes de a em G
 - Sem o *coalescing*, todos poderiam ser removidos, gerando um grafo G_1 .
 - Fazendo o *coalescing*, todos os vértices de S também poderão ser removidos, criando G_2 .
 - G_2 é um subgrafo de G_1 , onde o vértice ab corresponde ao b .

Coalescing

- George:
 - a e b podem ser unidos se para cada vizinho t de a :
 - t interfere com b .
 - ou t tem grau insignificante ($< k$)
 - Por que é segura?
 - Seja S o conjunto de vizinhos insignificantes de a em G
 - Sem o *coalescing*, todos poderiam ser removidos, gerando um grafo G_1 .
 - Fazendo o *coalescing*, todos os vértices de S também poderão ser removidos, criando G_2 .
 - G_2 é um subgrafo de G_1 , onde o vértice ab corresponde ao b .
 - G_2 é no mínimo tão fácil para colorir quanto G_1 .

Coalescing

- São estratégias conservativas

Coalescing

- São estratégias conservativas
- Podem sobrar operações de cópia que poderiam ser removidas.

Coalescing

- São estratégias conservativas
- Podem sobrar operações de cópia que poderiam ser removidas.
- Ainda assim, é melhor que fazer *spill*!.

Fases da Alocação com Coalescing

- Build:

Fases da Alocação com Coalescing

- Build:
 - Construir o IG.

Fases da Alocação com Coalescing

- Build:
 - Construir o IG.
 - Categorizar os vértices em *move-related* e *move-unrelated*.

Fases da Alocação com Coalescing

- Build:
 - Construir o IG.
 - Categorizar os vértices em *move-related* e *move-unrelated*.
- Simplify:

Fases da Alocação com Coalescing

- Build:
 - Construir o IG.
 - Categorizar os vértices em *move-related* e *move-unrelated*.
- Simplify:
 - Remover os vértices não significativos ($\text{grau} < k$) que são *move-unrelated*, um de cada vez.

Fases da Alocação com Coalescing

- Build:
 - Construir o IG.
 - Categorizar os vértices em *move-related* e *move-unrelated*.
- Simplify:
 - Remover os vértices não significativos ($\text{grau} < k$) que são *move-unrelated*, um de cada vez.
- Coalesce:

Fases da Alocação com Coalescing

- Build:
 - Construir o IG.
 - Categorizar os vértices em *move-related* e *move-unrelated*.
- Simplify:
 - Remover os vértices não significativos ($\text{grau} < k$) que são *move-unrelated*, um de cada vez.
- Coalesce:
 - Faça o coalesce conservativo no grafo resultante do passo anterior.

Fases da Alocação com Coalescing

- Build:
 - Construir o IG.
 - Categorizar os vértices em *move-related* e *move-unrelated*.
- Simplify:
 - Remover os vértices não significativos ($\text{grau} < k$) que são *move-unrelated*, um de cada vez.
- Coalesce:
 - Faça o coalesce conservativo no grafo resultante do passo anterior.
 - Com a redução dos graus, é provável que apareça mais oportunidades para o *coalescing*.

Fases da Alocação com Coalescing

- Build:
 - Construir o IG.
 - Categorizar os vértices em *move-related* e *move-unrelated*.
- Simplify:
 - Remover os vértices não significativos ($\text{grau} < k$) que são *move-unrelated*, um de cada vez.
- Coalesce:
 - Faça o coalesce conservativo no grafo resultante do passo anterior.
 - Com a redução dos graus, é provável que apareça mais oportunidades para o *coalescing*.
 - Quando um vértice resultante não é mais *move-related* ele fica disponível para a próxima simplificação.

Fases da Alocação com Coalescing

- Freeze:

Fases da Alocação com Coalescing

- Freeze:
 - Executado quando nem o simplify nem o coalescing podem ser aplicados.

Fases da Alocação com Coalescing

- Freeze:
 - Executado quando nem o simplify nem o coalescing podem ser aplicados.
 - Procura vértices *move-related* de grau baixo.

Fases da Alocação com Coalescing

- Freeze:
 - Executado quando nem o simplify nem o coalescing podem ser aplicados.
 - Procura vértices *move-related* de grau baixo.
 - Congela os moves desses vértices. Eles passam a ser candidatos para simplificação.

Fases da Alocação com Coalescing

- Freeze:
 - Executado quando nem o simplify nem o coalescing podem ser aplicados.
 - Procura vértices *move-related* de grau baixo.
 - Congela os moves desses vértices. Eles passam a ser candidatos para simplificação.
- Spill:

Fases da Alocação com Coalescing

- Freeze:
 - Executado quando nem o simplify nem o coalescing podem ser aplicados.
 - Procura vértices *move-related* de grau baixo.
 - Congela os moves desses vértices. Eles passam a ser candidatos para simplificação.
- Spill:
 - Se não houver vértices de grau baixo, seleciona-se um vértice com grau significativo para *spill*.

Fases da Alocação com Coalescing

- Freeze:
 - Executado quando nem o simplify nem o coalescing podem ser aplicados.
 - Procura vértices *move-related* de grau baixo.
 - Congela os moves desses vértices. Eles passam a ser candidatos para simplificação.
- Spill:
 - Se não houver vértices de grau baixo, seleciona-se um vértice com grau significativo para *spill*.
 - Coloca-se esse vértice na pilha.

Fases da Alocação com Coalescing

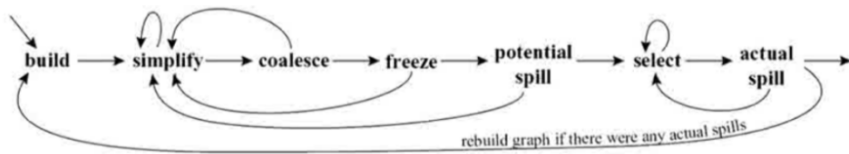
- Freeze:
 - Executado quando nem o simplify nem o coalescing podem ser aplicados.
 - Procura vértices *move-related* de grau baixo.
 - Congela os moves desses vértices. Eles passam a ser candidatos para simplificação.
- Spill:
 - Se não houver vértices de grau baixo, seleciona-se um vértice com grau significativo para *spill*.
 - Coloca-se esse vértice na pilha.
- Select:

Fases da Alocação com Coalescing

- Freeze:
 - Executado quando nem o simplify nem o coalescing podem ser aplicados.
 - Procura vértices *move-related* de grau baixo.
 - Congela os moves desses vértices. Eles passam a ser candidatos para simplificação.
- Spill:
 - Se não houver vértices de grau baixo, seleciona-se um vértice com grau significativo para *spill*.
 - Coloca-se esse vértice na pilha.
- Select:
 - Desempilhar todos os vértices e atribuir cores.

Fluxo com Coalescing

Simplify, coalesce e spill são intercalados até que o grado esteja vazio.



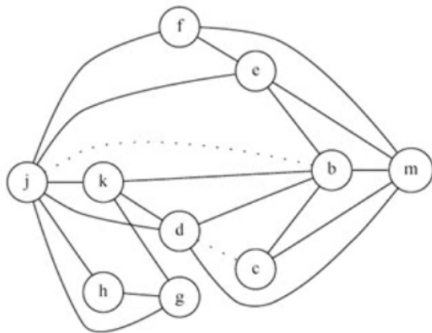
Retomando o Exemplo

- Suponha que temos 4 registradores.

Retomando o Exemplo

- Suponha que temos 4 registradores.
- Agora somente vértices não relacionados a cópias podem ser candidatos no *simplify*.

```
live-in: k j
g := mem[j+12]
h := k - 1
f := g * h
e := mem[j+8]
m := mem[j+16]
b := mem[f]
c := e + 8
d := c
k := m + 4
j := b
live-out: d k j
```

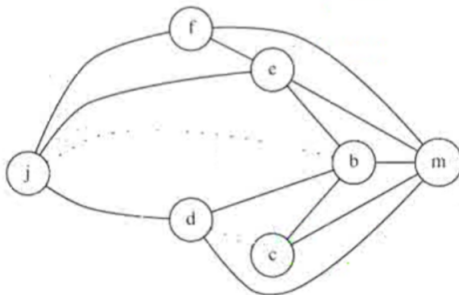


Exemplo de coalescing

- Removendo h , g , k .

Exemplo de coalescing

- Removendo h, g, k .
- Invocando coalescing....



Exemplo de coalescing

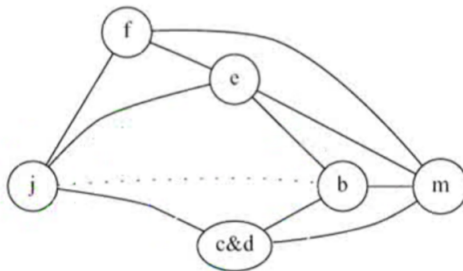
- c e d podem ser unidos.

Exemplo de coalescing

- c e d podem ser unidos.
 - $c \& d$ tem 1 vizinho com grau significativo ($\geq k$).

Exemplo de coalescing

- c e d podem ser unidos.
 - $c\&d$ tem 1 vizinho com grau significativo ($\geq k$).



Exemplo de coalescing

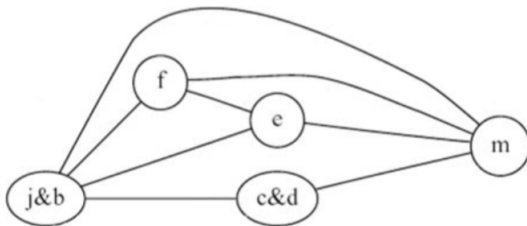
- b e j também podem ser unidos.

Exemplo de coalescing

- b e j também podem ser unidos.
 - $j \& b$ tem 1 vizinho com grau significativo ($\geq k$).

Exemplo de coalescing

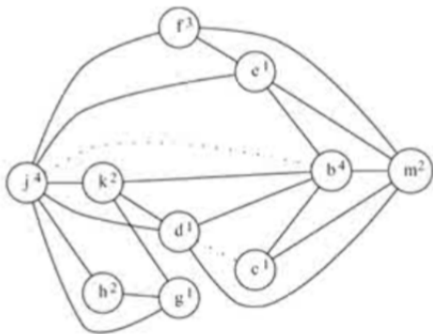
- b e j também podem ser unidos.
 - $j\&b$ tem 1 vizinho com grau significativo ($\geq k$).
- Agora *simplify* termina o trabalho...



Exemplo de coalescing

- Alocação final

e	1
m	2
f	3
j&b	4
c&d	1
k	2
h	2
g	1
stack	coloring



Pré-coloração

- Alguns vértices do IG podem ser pré-coloridos.

Pré-coloração

- Alguns vértices do IG podem ser pré-coloridos.
 - Temporários associados ao FP, SP, registradores de passagem de argumentos.

Pré-coloração

- Alguns vértices do IG podem ser pré-coloridos.
 - Temporários associados ao FP, SP, registradores de passagem de argumentos.
 - Permanentemente associados aos registradores físicos.

Pré-coloração

- Alguns vértices do IG podem ser pré-coloridos.
 - Temporários associados ao FP, SP, registradores de passagem de argumentos.
 - Permanentemente associados aos registradores físicos.
 - Cores pré-definidas e únicas.

Pré-coloração

- Alguns vértices do IG podem ser pré-coloridos.
 - Temporários associados ao FP, SP, registradores de passagem de argumentos.
 - Permanentemente associados aos registradores físicos.
 - Cores pré-definidas e únicas.
 - Podem ser reaproveitados no *select* e *coalesce*.

Pré-coloração

- Alguns vértices do IG podem ser pré-coloridos.
 - Temporários associados ao FP, SP, registradores de passagem de argumentos.
 - Permanentemente associados aos registradores físicos.
 - Cores pré-definidas e únicas.
 - Podem ser reaproveitados no *select* e *coalesce*.
 - Desde que não interfiram com o outro valor.

Pré-coloração

- Alguns vértices do IG podem ser pré-coloridos.
 - Temporários associados ao FP, SP, registradores de passagem de argumentos.
 - Permanentemente associados aos registradores físicos.
 - Cores pré-definidas e únicas.
 - Podem ser reaproveitados no *select* e *coalesce*.
 - Desde que não interfiram com o outro valor.
 - Ex.: um registrador de passagem de parâmetro pode servir como temporário no corpo da função.

Pré-coloração

- Podem ser unidos no calescing com outros vértices não pré-coloridos.

Pré-coloração

- Podem ser unidos no calescing com outros vértices não pré-coloridos.
- Simplify os trata como tendo grau “infinito”.

Pré-coloração

- Podem ser unidos no calescing com outros vértices não pré-coloridos.
- Simplify os trata como tendo grau “infinito”.
 - Não devem ir para a pilha.

Pré-coloração

- Podem ser unidos no coalescing com outros vértices não pré-coloridos.
- Simplify os trata como tendo grau “infinito”.
 - Não devem ir para a pilha.
 - Não devem sofrer spill.

Pré-coloração

- Podem ser unidos no coalescing com outros vértices não pré-coloridos.
- Simplify os trata como tendo grau “infinito”.
 - Não devem ir para a pilha.
 - Não devem sofrer spill.
 - O algoritmo executa *simplify*, *select* e *spill* até sobrarem somente vértices pré-coloridos.

Pré-coloração

- Podem ser copiados para temporários

Pré-coloração

- Podem ser copiados para temporários
 - Suponha que *r7* seja um *callee-save register*.

$$\begin{array}{l} \text{enter: } \text{def}(r_7) \\ (a) \quad \vdots \\ \text{exit: } \text{use}(r_7) \end{array}$$
$$\begin{array}{lcl}
 & \text{enter:} & \text{def}(r_7) \\
 & & t_{231} \leftarrow r_7 \\
 (b) & & \vdots \\
 & & r_7 \leftarrow t_{231} \\
 & \text{exit:} & \text{use}(r_7)
 \end{array}$$

Exemplo

- Três registradores

Exemplo

- Três registradores
 - $r1$ e $r2$ *caller-save*.

Exemplo

- Três registradores
 - $r1$ e $r2$ *caller-save*.
 - $r3$ *callee-save*.

```
int f(int a, int b) {  
    int d=0;  
    int e=a;  
    do {d = d+b;  
        e = e-1;  
    } while (e>0);  
    return d;  
}
```

(a)

```
enter:  c ← r3  
        a ← r1  
        b ← r2  
        d ← 0  
        e ← a  
loop:  d ← d + b  
        e ← e - 1  
        if e > 0 goto loop  
        r1 ← d  
        r3 ← c  
return (r1, r3 live out)
```

(b)

Exemplo

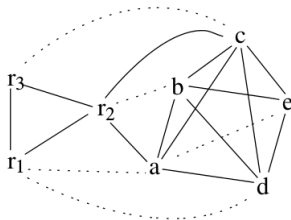
- IG para o programa em (b)

Exemplo

- IG para o programa em (b)
 - $K = 3$

Exemplo

- IG para o programa em (b)
 - $K = 3$
 - Tem oportunidade para simplify e spill?



Prioridade de Spill

Nesse grafo, não temos oportunidade para *simplify* ou *freeze*, pois todos os vértices pré-coloridos possuem grau $\geq k$. Portanto, devemos escolher algum vértice como *spill*.

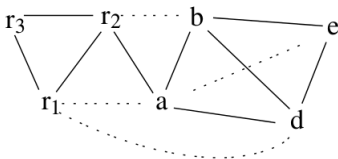
Node	Uses+Defs outside loop				Uses+Defs within loop				Degree	Spill priority	
<i>a</i>	(	2	+	10	×	0	)	/	4	=	0.50
<i>b</i>	(	1	+	10	×	1	)	/	4	=	2.75
<i>c</i>	(	2	+	10	×	0	)	/	6	=	0.33
<i>d</i>	(	2	+	10	×	2	)	/	4	=	5.50
<i>e</i>	(	1	+	10	×	3	)	/	3	=	10.33

Exemplo

- O vértice c possui a menor prioridade - ele interfere com muitos outros temporários, mas é raramente usado.

Exemplo

- O vértice c possui a menor prioridade - ele interfere com muitos outros temporários, mas é raramente usado.
 - Fazendo o *spill* de c , obtemos:

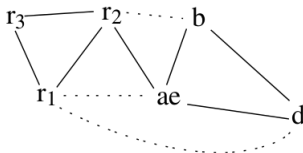


Exemplo

- Podemos agora fazer o *coalesce* de a e e , tendo como resultado um vértice adjacente com menos que k vértices com grau significativo

Exemplo

- Podemos agora fazer o *coalesce* de a e e , tendo como resultado um vértice adjacente com menos que k vértices com grau significativo
 - Fazendo o *coalesce* de a e e , obtemos:

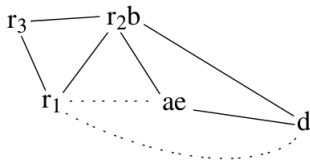


Exemplo

- Agora podemos fazer o *coalesce* de $a \& r_1$ ou de $b \& r_2$.

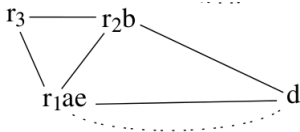
Exemplo

- Agora podemos fazer o *coalesce* de $ae \& r_1$ ou de $b \& r_2$.
 - Fazendo o *coalesce* de $b \& r_2$, obtemos:



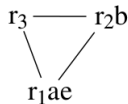
Exemplo

- Fazendo o *coalesce* de ae & r_1 , obtemos:



Exemplo

- Não podemos fazer *coalesce* se $r_1ae \& d$ porque o move é *constrained*: os vértices r_1ae e d interferem entre si.
 - Simplificando d , temos:



Exemplo

- Agora chegamos a um grafo com apenas o vértices pré-coloridos. Assim, podemos desempilhar os vértices da pilha e atribuir cor aos mesmos.

Exemplo

- Agora chegamos a um grafo com apenas os vértices pré-coloridos. Assim, podemos desempilhar os vértices da pilha e atribuir cor aos mesmos.
- Ao desempilharmos o d , podemos atribuir a cor r_3 a ele.

Exemplo

- Agora chegamos a um grafo com apenas o vértices pré-coloridos. Assim, podemos desempilhar os vértices da pilha e atribuir cor aos mesmos.
- Ao desempilharmos o d , podemos atribuir a cor r_3 a ele.
- Os vértices a , b e e já foram cores que foram atribuídas pelo *coalescing*.

Exemplo

- Agora chegamos a um grafo com apenas os vértices pré-coloridos. Assim, podemos desempilhar os vértices da pilha e atribuir cor aos mesmos.
- Ao desempilharmos o d , podemos atribuir a cor r_3 a ele.
- Os vértices a , b e e já foram cores que foram atribuídas pelo *coalescing*.
- O vértice c , que foi marcado como um potencial *spill*, se tornou um *spill* de fato, pois quando é retirado da pilha não temos uma cor disponível para atribuir a ele.

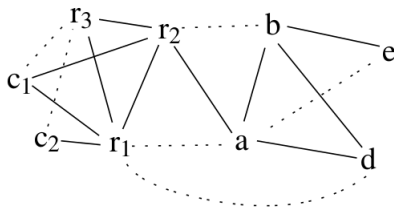
Exemplo

- Como tivemos que fazer *spilling* de c nessa rodada, devemos reescrever o programa para incluir as instruções de *spill*.
- Para cada uso (ou definição) de c , criamos um temporário e fazemos *fetch* (ou *store*) dele imediatamente antes (ou depois).

```
enter:   $c_1 \leftarrow r_3$   
         $M[c_{\text{loc}}] \leftarrow c_1$   
         $a \leftarrow r_1$   
         $b \leftarrow r_2$   
         $d \leftarrow 0$   
         $e \leftarrow a$   
loop:    $d \leftarrow d + b$   
         $e \leftarrow e - 1$   
        if  $e > 0$  goto loop  
         $r_1 \leftarrow d$   
         $c_2 \leftarrow M[c_{\text{loc}}]$   
         $r_3 \leftarrow c_2$   
        return
```

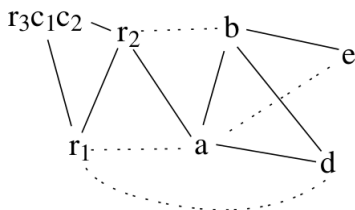
Exemplo

- Feito isso, precisamos construir um novo grafo de interferência.



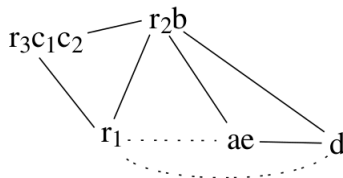
Exemplo

- Podemos fazer o *coalesce* de $c_1 \& r_3$ e então de $c_2 \& r_3$.



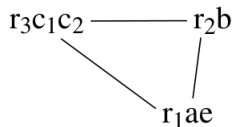
Exemplo

- Agora podemos fazer o coalesce de $a \& e$ e então de $b \& r_2$.



Exemplo

- E finalmente, podemos fazer o *coalesce* de $ae \& r_1$ e então fazer *simplify* de d .



Exemplo

- Agora podemos começar a desempilhar os vértices da pilha.
- Podemos atribuir a cor r_3 para d , e esse era o único vértice que foi para pilha.
- Todos os outros foram *coalesced* ou pré-coloridos.

Node	Color
a	r_1
b	r_2
c	r_3
d	r_3
e	r_1

Exemplo

- Agora podemos reescrever o programa usando a coloração feita.

```
enter:   $r_3 \leftarrow r_3$   
         $M[c_{\text{loc}}] \leftarrow r_3$   
         $r_1 \leftarrow r_1$   
         $r_2 \leftarrow r_2$   
         $r_3 \leftarrow 0$   
         $r_1 \leftarrow r_1$   
loop:   $r_3 \leftarrow r_3 + r_2$   
         $r_1 \leftarrow r_1 - 1$   
        if  $r_1 > 0$  goto loop  
         $r_1 \leftarrow r_3$   
         $r_3 \leftarrow M[c_{\text{loc}}]$   
         $r_3 \leftarrow r_3$   
        return
```


Exemplo

- Finalmente, podemos remover qualquer instrução de MOVE onde o endereço fonte e destino são o mesmo.
- Obtermos o seguinte código:

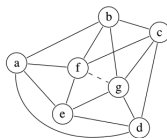
```
enter:   $M[c_{loc}] \leftarrow r_3$   
         $r_3 \leftarrow 0$   
loop:    $r_3 \leftarrow r_3 + r_2$   
         $r_1 \leftarrow r_1 - 1$   
        if  $r_1 > 0$  goto loop  
         $r_1 \leftarrow r_3$   
         $r_3 \leftarrow M[c_{loc}]$   
        return
```

Algoritmo

```
procedure Main()
    LivenessAnalysis()
    Build()
    MakeWorklist()
    repeat
        if simplifyWorklist  $\neq$  {} then Simplify()
        else if worklistMoves  $\neq$  {} then Coalesce()
        else if freezeWorklist  $\neq$  {} then Freeze()
        else if spillWorklist  $\neq$  {} then SelectSpill()
    until simplifyWorklist = {}  $\wedge$  worklistMoves = {}
         $\wedge$  freezeWorklist = {}  $\wedge$  spillWorklist = {}
    AssignColors()
    if spilledNodes  $\neq$  {} then
        RewriteProgram(spilledNodes)
    Main()
```

Exercício

Coalescing conservativos são assim chamados por não introduzem *spill*. Mas eles podem evitar *spill*? considere o seguinte grafo, onde as arestas normais representam interferências e as arestas tracejadas representam um MOVE:



- 1 Faça uma 4-coloração sem usar *coalescing*. Mostre a ordem em que os vértices foram removidos. Existe algum potencial *spill*? Ele é um *spill*?
- 2 Faça uma 4-coloração usando *coalescing*. Você usou o critério de Briggs ou de George? Existem algum potencial *spill*? Ele é um *spill*?

Alocação de Registradores



Universidade Federal do Ceará - Campus de Quixadá

Lucas Ismaily
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)